

Revisi dan hasil perbaikan setelah test lisan

No	Nama	Tugas perbaikan saat test lisan
1	Fitri Nur Aini (25032014080)	Memperbaiki logika fungsi pembaruan rata-rata pada K-Means dasar. Menambahkan kondisi agar nilai <i>centroid</i> sebelumnya dipertahankan jika klaster tidak memiliki anggota pada iterasi tersebut.
Kode Program		
<pre>def kmeans_dasar(X, k, centroid_awal, fungsi_jarak, nilai_p=3, max_iter=100, toleransi=1e-4): centroids = centroid_awal.copy() for iterasi in range(max_iter): # Hitung jarak dan tentukan anggota masing-masing klaster distances = fungsi_jarak(X, centroids, p=nilai_p) labels = np.argmin(distances, axis=1) old_centroids = centroids.copy() new_centroids = [] for i in range(k): # Ambil semua anggota yang masuk ke klaster ke-i points = X[labels == i] if len(points) > 0: # Jika klaster ada isinya, maka rata-rata dihitung murni # dari anggota klaster tersebut. new_centroids.append(np.mean(points, axis=0)) else: # Jika klaster kosong # Nilai centroid sebelumnya dibiarkan dan ikut dihitung # sebagai centroid baru. new_centroids.append(centroids[i]) centroids = np.array(new_centroids) # Hentikan iterasi jika centroid sudah konvergen if np.allclose(old_centroids, centroids, atol=toleransi): break return centroids, labels</pre>		
2	Camellia Mardatilla Wirantono (25032014051)	Memperbaiki fungsi perhitungan rata-rata centroids baru pada algoritma incremental k-means
Kode		
<pre>def incremental_kmeans(X, threshold_T, fungsi_jarak_titik, fungsi_jarak_semua, nilai_p=3, max_iter=100, toleransi=1e-4):</pre>		

```

centroids_list = [X[0]]
labels = np.zeros(len(X), dtype=int)

for i in range(1, len(X)):
    titik = X[i]
    jarak_ke_centroids = fungsi_jarak_titik(titik,
np.array(centroids_list), p=nilai_p)
    jarak_minimum = np.min(jarak_ke_centroids)

    if jarak_minimum > threshold_T:
        centroids_list.append(titik)
        labels[i] = len(centroids_list) - 1
    else:
        labels[i] = np.argmin(jarak_ke_centroids)

centroids = np.array(centroids_list)
k_terbentuk = len(centroids)

for iterasi in range(max_iter):
    distances = fungsi_jarak_semua(X, centroids, p=nilai_p)
    labels = np.argmin(distances, axis=1)
    old_centroids = centroids.copy()

    new_centroids = []
    for c in range(k_terbentuk):
        points = X[labels == c]
        if len(points) > 0:
            old_c = centroids[c] #fungsi untuk include centroid lama
kedalam fungsi perhitungan centroid baru
            points_and_old_c = np.vstack([points, old_c])
            new_centroids.append(np.mean(points_and_old_c, axis=0))
#perhitungan centroid baru dengan memasukkan bobot centroid lama
        else:
            new_centroids.append(centroids[c])

    centroids = np.array(new_centroids)

    if np.allclose(old_centroids, centroids, atol=toleransi):
        break

return centroids, labels

```

3	Salsabila Arsa Azura (25032014085)	Membuat program K-Means menggunakan metode Manhattan Distance pada 10 data pertama dataset Iris serta melakukan perhitungan manual untuk memverifikasi hasil clustering yang diperoleh dari program.
---	---------------------------------------	--

kode

```

from sklearn.datasets import load_iris
import pandas as pd
import numpy as np

# Memuat dataset Iris
iris = load_iris()

data = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

# Mengambil 10 data pertama
data = data.iloc[:10]

# Mengubah indeks agar dimulai dari 1
data_tampil = data.copy()
data_tampil.index = range(1, len(data_tampil) + 1)

print("10 Data Pertama Dataset Iris")
print(data_tampil)

# Menyiapkan data untuk K-Means
X = data.values

# Jumlah cluster
K = 2

# Centroid awal menggunakan data ke-1 dan data ke-2
centroids = np.array([
    X[0],
    X[1]
])

print("\nCentroid Awal")

for i, c in enumerate(centroids):

```

```

    print(f"C{i+1} = {c}")

iterasi = 1

while True:

    print(f"\nIterasi {iterasi}")

    hasil_cluster = []
    hasil_tabel = []

    # Menghitung Manhattan Distance
    for i, titik in enumerate(X):

        d1 = np.sum(np.abs(titik - centroids[0]))
        d2 = np.sum(np.abs(titik - centroids[1]))

        if d1 < d2:
            cluster = 1
        else:
            cluster = 2

        hasil_cluster.append(cluster - 1)

        hasil_tabel.append([
i + 1,
round(d1, 2),
round(d2, 2),
cluster
])

    # Menampilkan hasil iterasi
    df_hasil = pd.DataFrame(
        hasil_tabel,
        columns=[
            "Data",
            "Jarak C1",
            "Jarak C2",
            "Cluster"
        ]
    )

```

```

print(df_hasil)

hasil_cluster = np.array(hasil_cluster)

# Menyimpan centroid lama
centroid_lama = centroids.copy()

# Menghitung centroid baru
centroid_baru = []

for k in range(K):

    anggota = X[hasil_cluster == k]

    centroid_baru.append(
        np.mean(anggota, axis=0)
    )

centroids = np.array(centroid_baru)

print("\nCentroid Baru")

for i, c in enumerate(centroids):
    print(f"C{i+1} = {np.round(c, 4)}")

# Mengecek apakah centroid sudah stabil
if np.allclose(centroid_lama, centroids):

    print("\nCentroid sudah stabil.")
    print("Proses K-Means selesai.")

    break

iterasi += 1

print("\nHasil Akhir")

hasil_akhir = pd.DataFrame({
    "Data": range(1, len(X) + 1),
    "Cluster Final": hasil_cluster + 1
})

```

```

print(hasil_akhir)

print("\nCentroid Final")

for i, c in enumerate(centroids):
    print(f"C{i+1} = {np.round(c, 4)}")

```

No. _____
Date: _____

PERHITUNGAN MANUAL MANHATTAN DISTANCE

Rumus Manhattan $D = |x_1 - c_1| + |x_2 - c_2| + |x_3 - c_3| + |x_4 - c_4|$

Data ke-3 = (4.7, 3.2, 1.3, 0.2)

centroid 1 = (5.1, 3.5, 1.4, 0.2)

centroid 2 = (4.9, 3.0, 1.4, 0.2)

$D(c_1) = |4.7 - 5.1| + |3.2 - 3.5| + |1.3 - 1.4| + |0.2 - 0.2|$
 $= 0.4 + 0.3 + 0.1 + 0$
 $= 0.8$

$D(c_2) = |4.7 - 4.9| + |3.2 - 3.0| + |1.3 - 1.4| + |0.2 - 0.2|$
 $= 0.2 + 0.2 + 0.1 + 0$
 $= 0.5$

Karena $D(c_2) < D(c_1)$
 $0.5 < 0.8$

Maka data ke-3 masuk cluster 2.

Big BOSS